

SecureSplit: Mitigating Backdoor Attacks in Split Learning

Zhihao Dou*
Case Western Reserve University
Cleveland, USA

Dongfei Cui*
Northeast Electric Power University
Jilin, China

Weida Wang*
Fudan University
Shanghai, China

Anjun Gao
University of Louisville
Louisville, USA

Yueyang Quan
University of North Texas
Denton, USA

Mengyao Ma
The University of Queensland
Brisbane, Australia

Viet Vo
Swinburne University of Technology
Melbourne, Australia

Guangdong Bai
City University of Hong Kong
Hong Kong, China

Zhuqing Liu
University of North Texas
Denton, USA

Minghong Fang
University of Louisville
Louisville, USA

Abstract

Split Learning (SL) offers a framework for collaborative model training that respects data privacy by allowing participants to share the same dataset while maintaining distinct feature sets. However, SL is susceptible to backdoor attacks, in which malicious clients subtly alter their embeddings to insert hidden triggers that compromise the final trained model. To address this vulnerability, we introduce SecureSplit, a defense mechanism tailored to SL. SecureSplit applies a dimensionality transformation strategy to accentuate subtle differences between benign and poisoned embeddings, facilitating their separation. With this enhanced distinction, we develop an adaptive filtering approach that uses a majority-based voting scheme to remove contaminated embeddings while preserving clean ones. Rigorous experiments across four datasets (CIFAR-10, MNIST, CINIC-10, and ImageNette), five backdoor attack scenarios, and seven alternative defenses confirm the effectiveness of SecureSplit under various challenging conditions.

CCS Concepts

• Security and privacy → Systems security.

Keywords

Split Learning; Backdoor Attacks; Robustness

ACM Reference Format:

Zhihao Dou, Dongfei Cui, Weida Wang, Anjun Gao, Yueyang Quan, Mengyao Ma, Viet Vo, Guangdong Bai, Zhuqing Liu, and Minghong Fang. 2026. SecureSplit: Mitigating Backdoor Attacks in Split Learning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792484>

*Equal contribution. Zhihao Dou conducted this research while he was an intern under the supervision of Minghong Fang.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792484>

United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792484>

1 Introduction

As data privacy concerns grow, federated learning (FL) [30] has gained attention for enabling collaborative model training without exposing raw data. FL allows multiple clients to train a shared model while keeping their data local, and is typically categorized into horizontal federated learning (HFL) [1, 7, 33, 47] and split learning (SL) [18, 26, 34, 36, 39, 40, 42]. HFL applies when clients share the same feature space but hold different samples; for example, banks collaboratively train a credit scoring model using similar features but from distinct user groups. In contrast, SL applies when each client holds only a subset of the features, while the server owns the labels. For example, one institution may store income data while another holds repayment records. Each client trains a *bottom model* to convert local features into embeddings, which are sent to the server. The server uses these embeddings and labels to update a *top model*, and then returns gradients to refine the clients' bottom models via backpropagation.

Although SL is decentralized, it remains vulnerable to poisoning attacks, particularly backdoor attacks [4, 19, 32]. In such attacks, adversaries control malicious clients that generate embeddings crafted to manipulate the top model. These clients embed backdoor triggers into their local data or transmitted embeddings, causing the top model to misclassify specific inputs into attacker-chosen labels. For instance, the VILLAIN attack [4] leverages label inference to locate target-class samples and subtly inserts a stealthy trigger into the embeddings of compromised clients. A common countermeasure is to adopt Byzantine-robust aggregation [5, 6, 14–16, 48] from HFL to mitigate backdoor attacks. However, our experiments show that directly applying these defenses in SL offers limited robustness. To enhance resilience, VFLIP [8] introduces identification and purification strategies. While it achieves partial mitigation, experimental results show that backdoor threats are not fully eliminated.

Our work: To strengthen resilience against backdoor attacks in SL, we introduce SecureSplit, a two-step defense strategy tailored to identify and neutralize poisoned embeddings in SL frameworks. Our

SecureSplit leverages an innovative embedding transformation pipeline combined with an adaptive majority-based filtering technique, strategically designed to enhance the separability between benign and poisoned embeddings. By systematically manipulating the embedding space, SecureSplit effectively identifies subtle yet critical discrepancies from adversarial manipulation, thereby significantly improving robustness against stealthy backdoor attacks.

In the first step, we implement a carefully structured dimensionality transformation approach composed of dimensionality reduction followed by dimensional expansion. Specifically, we first apply a dimensionality reduction technique to compress the original high-dimensional embedding space into a lower-dimensional representation. This step removes noisy dimensions while preserving local geometric relationships, ensuring that structural proximity between benign and poisoned embeddings remains intact and discernible. However, as dimensionality reduction might result in a loss of nuanced information crucial for distinguishing benign from poisoned embeddings, we subsequently apply a dimensional expansion method. This transformation reintroduces non-linear feature interactions, amplifying subtle differences and substantially enhancing separability.

Following the embedding transformation, our second step introduces an adaptive majority-based filtering method aimed at robustly identifying poisoned embeddings. Initially, we compute the coordinate-wise median of the transformed embeddings, establishing a robust central reference point indicative of benign embedding behavior. We then employ a majority-based radius determination that retains at least half of the embeddings closest to this median. To address potential misclassification issues and improve precision, we dynamically adjust this radius according to the variance observed within the embeddings: a lower variance leads to a more aggressive enlargement of the radius to capture additional benign embeddings, while higher variance dictates a conservative adjustment to maintain defense robustness. This adaptive strategy balances excluding poisoned embeddings and retaining benign ones, improving detection performance.

We rigorously evaluate our proposed SecureSplit across four diverse datasets (i.e., CIFAR-10 [22], MNIST [23], CINIC-10 [9], and ImageNette [20]), assessing its resilience against five types of backdoor attacks, including a strong adaptive attack. Additionally, we compare its performance against seven state-of-the-art SL baselines, encompassing extensions of standard HFL approaches such as Trimmed-mean [48] and Multi-Krum [5], clustering-based methods like HDBSCAN [28], and defenses specifically designed for the SL framework. The results demonstrate that our SecureSplit effectively enhances robustness and adaptability across a broad spectrum of real-world backdoor scenarios. The key contributions of this paper are summarized as follows:

- We present SecureSplit, an innovative defense designed to counteract backdoor attacks in split learning.
- We conduct a thorough empirical evaluation of SecureSplit in the face of various backdoor attacks, with our results confirming its capability to effectively defend against them.
- We craft an adaptive attack against SecureSplit and evaluate its impact, revealing that SecureSplit maintains its resilience even under this targeted threat.

2 Preliminaries and Related Work

2.1 Background on Split Learning

Collaborative learning frameworks like federated learning (FL) allow multiple clients to train models without sharing raw data. While preserving privacy, approaches differ in data distribution. In horizontal federated learning (HFL) [1, 7, 33, 47], each client holds labeled data with a shared feature space. In contrast, SL [18, 26, 36] operates under a vertically partitioned data setting, where clients hold different subsets of features for the entire training dataset and do not have access to labels, which are exclusively maintained by a central server. Formally, consider a SL system with n clients, each denoted as C_i . Let $\mathbf{x}_k$ be the complete feature vector of a training example k , represented as $\mathbf{x}_k = [x_{k,1}, x_{k,2}, \dots, x_{k,z}]$, where z is the total number of features. In SL, each client C_i holds only a subset of the features, denoted as $\hat{\mathbf{x}}_k^i$. The full feature vector is distributed across all clients such that $\mathbf{x}_k = \bigcup_{i=1}^n \hat{\mathbf{x}}_k^i$. Unlike FL, where clients manage both features and labels, in SL, the label y_k for each training example is stored solely on the server. In each training round t , SL follows three fundamental steps. For clarity, we omit t in the subsequent explanation.

- **Step I (Feature embeddings generation):** Each client C_i processes its local feature subset using a bottom model $\mathcal{L}_i$, generating an embedding vector:

$$E_k^i = \mathcal{L}_i(\hat{\mathbf{x}}_k^i), \quad (1)$$

where E_k^i is the embedding for training example k from client C_i , d is the dimension of E_k^i . Clients then send their embeddings to the server for further processing.

- **Step II (Embeddings aggregation):** Upon receiving embedding vectors from the clients, the server applies an aggregation function $\mathcal{A}(\cdot)$ to merge them as follows:

$$E_k = \mathcal{A}(E_k^1, E_k^2, \dots, E_k^n). \quad (2)$$

This aggregated embedding $E_k \in \mathbb{R}^d$, along with its corresponding label y_k , is then used to train a top model in a supervised learning framework. Once the model updates are computed, the server calculates the gradient $\mathbf{g}_i$ of the loss with respect to each client's embedding E_k^i . Finally, the server transmits $\mathbf{g}_i$ back to the respective client C_i for updating their bottom models.

- **Step III (Bottom models updating):** Each client C_i updates the parameters of its bottom model $\mathcal{L}_i$ using the received gradient $\mathbf{g}_i$. This process allows each client to refine its bottom model using server feedback, leading to improved feature representations in future rounds.

Note that for simplicity, we assume that each client transmits the embedding of a single training example to the server in the three steps outlined above. However, in practice, clients in SL send embeddings of multiple training examples to the server, which can be seamlessly extended from the single-example transmission process. We note that SL differs fundamentally from both horizontal federated learning (HFL) and vertical federated learning (VFL), see Appendix A for details.

2.2 Backdoor Attacks to SL

SL is a decentralized learning framework where clients share embeddings with the server during each training round. However, this collaborative structure and frequent communication between clients and the server create vulnerabilities, making SL susceptible to backdoor attacks [4, 19, 32]. These attacks enable the attacker to manipulate the learned top model, causing it to produce attacker-specified outputs for selected inputs. To execute a backdoor attack, a malicious client can introduce hidden triggers into its training data, subtly poisoning the learning process. For example, in Bad-VFL attack [32], the attacker embeds triggers directly into the local datasets of malicious clients, influencing the top model’s behavior. Beyond data-level poisoning, the attacker can manipulate embeddings to stealthily implant backdoors. The VILLAIN attack [4] exemplifies this strategy, where the attacker crafts malicious embeddings while fine-tuning the trigger’s magnitude to maximize impact while minimizing detectability.

2.3 Defenses against Backdoor Attacks to SL

To defend against backdoor attacks in SL, a common strategy is to adapt Byzantine-robust aggregation from FL [5, 6, 10, 12–16, 31, 43, 46, 48, 49]. For example, Trimmed-mean [48] removes extreme values in each embedding dimension before averaging, while clustering-based methods like HDBSCAN [28] group embeddings to filter out potential threats. However, as shown in our experiments, these methods are largely ineffective against sophisticated SL-specific attacks. More recent defenses include VFLIP [32], which uses identification and purification techniques but suffers from high false positives, often misclassifying benign embeddings. Another recent work [35] proposed a defense specifically designed for U-shaped SL, where clients are trained sequentially. Note that here we mean the training order among clients rather than the operation order between the clients and the server. As a result, the method in [35] is not applicable to our standard SL setting, where clients train in parallel. In Section 5.2, we adapt our method to the U-shaped SL setting and compare it with [35]; the results are shown in Table 7 in the Appendix.

3 Problem Statement

Threat model: Building on the threat model from prior work [4, 17, 19], we assume an attacker controlling one or more malicious clients. These clients can introduce backdoor triggers by poisoning local data or embedding manipulated triggers into their transmitted embeddings. As shown in Fig. 1, such embeddings cause the top model to produce attacker-specified outputs. Consistent with existing SL backdoor attacks [4, 19, 32], we consider a sophisticated attacker with access to limited labeled auxiliary data, enabling them to infer training labels and craft more precise attacks. Note that in our threat model, we assume the majority of training data are benign. This assumption is widely adopted in the machine learning and FL communities [11], as the attacker aims to remain stealthy. If most training data were malicious, the attacker would be easily detected, violating the covert-adversary assumption.

Defender’s knowledge and goal: Our goal is to design a robust defense against backdoor attacks in SL, despite the server receiving only client embeddings without access to their data, models, or

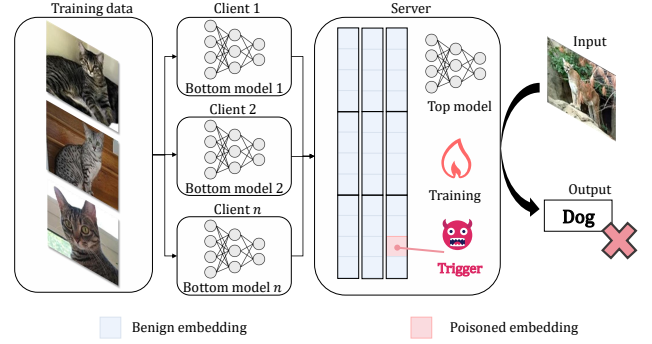


Figure 1: Split learning under backdoor attack.

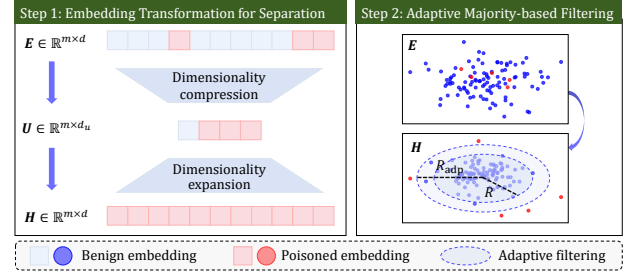


Figure 2: Illustration of our SecureSplit.

attack strategies. Such a defense must minimally affecting model performance when no attacks occur, and effectively identifying and excluding poisoned embeddings to maintain high accuracy and drastically reduce attack success rates.

4 The SecureSplit Algorithm

4.1 Overview

Our SecureSplit enhances the robustness of SL against backdoor attacks through two main steps: embedding transformation and adaptive filtering. The transformation restructures the embedding space to amplify subtle differences introduced by backdoor triggers. The adaptive filter then removes suspicious embeddings based on majority consensus, dynamically adjusting its criteria to reduce misclassification. Together, these steps effectively mitigate backdoor attacks while preserving model performance. Fig. 2 illustrates the overall framework. We next detail how SecureSplit operates in each training round.

4.2 Embedding Transformation for Separation

Consider a dataset consisting of m training examples, where each example is represented by a feature embedding. We denote the full set of embeddings as $E = \{E_1, E_2, \dots, E_m\}$, where each individual embedding E_k (for $k = 1, 2, \dots, m$) is obtained according to Eq. (2). Since each embedding is a d -dimensional vector, we have $E \in \mathbb{R}^{m \times d}$. In a SL setting, backdoor attacks involve injecting triggers into the embeddings of malicious clients, altering their representation. As a result, E may contain both benign embeddings (E_{Ben}) and poisoned embeddings (E_{Poi}). While the modifications introduced by the attacker are often subtle, they can still alter the spatial distribution of embeddings, shifting the positional relationships between poisoned and benign samples.

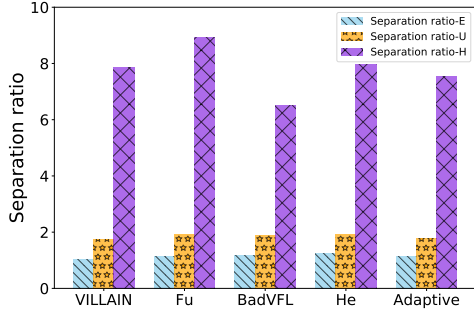


Figure 3: Separation ratio on different embedding sets.

To address this challenge, we first apply dimensionality compression using Uniform Manifold Approximation and Projection (UMAP) [29], transforming the original embedding space into a lower-dimensional representation. The compressed embedding set $U \in \mathbb{R}^{m \times d_u}$ is obtained as:

$$U = \text{UMAP}(E), \quad (3)$$

where $U = \{\hat{E}_1, \hat{E}_2, \dots, \hat{E}_m\}$, each embedding $\hat{E}_k$ (for $k = 1, 2, \dots, m$) is now in a reduced-dimensional space $\mathbb{R}^{d_u}$ with $d_u < d$. The motivation for dimensionality reduction is twofold. First, UMAP is designed to preserve the local geometric structure of data, ensuring that the positional relationships (e.g., relative proximity) between E_{Ben} and E_{Poi} in the original space are maintained in U . Second, by reducing dimensions, UMAP filters out noisy features that obscure key differences, enhancing the separation of poisoned and benign embeddings while preserving important structural patterns.

However, while dimensionality reduction enhances local relationships, it may also cause a loss of fine-grained information that is useful for separating benign and poisoned embeddings. To compensate for this, we introduce a dimensionality expansion step using Polynomial Kernel Transformation (PKT) [44], which maps the compressed embeddings back to a higher-dimensional space, restoring and amplifying crucial variations. The expanded embedding set $H \in \mathbb{R}^{m \times d}$ is obtained as follows:

$$H = \text{PKT}(U), \quad (4)$$

where $H = \{\bar{E}_1, \bar{E}_2, \dots, \bar{E}_m\}$, with each $\bar{E}_k$ is in the restored high-dimensional space $\mathbb{R}^d$. The PKT expansion introduces nonlinear transformations that enhance the separation between benign and poisoned embeddings by leveraging higher-order feature interactions. This step is crucial as it amplifies subtle distance variations, making anomalies more distinguishable and easier to detect.

In summary, our SecureSplit strategically combines dimensionality reduction and expansion to maximize the separability of poisoned and benign embeddings. First, UMAP removes noise dimensions while retaining the core structural relationships between embeddings. Then, PKT reintroduces higher-dimensional representations that emphasize and enhance these relationships, allowing for a more distinct separation between poisoned and benign embeddings in the final space. This two-step approach provides a robust mechanism for detecting backdoor attacks by making poisoned embeddings more distinguishable from their benign counterparts.

Empirical analysis: To validate SecureSplit, we conduct experiments on CIFAR-10, showing that dimensionality reduction and

Algorithm 1 SecureSplit.

Input: Full set of embeddings E , parameter α .

Output: Benign embedding set S .

```

1:  $S \leftarrow \emptyset$ .
2: // Step I: Embedding Transformation for Separation.
3: Compute compressed embeddings  $U$  using Eq. (3).
4: Compute expanded embeddings  $H$  per Eq. (4).
5: // Step II: Adaptive Majority-based Filtering.
6: Calculate coordinate-wise median:  $\Lambda \leftarrow \text{Median}(H)$ .
7: Determine radius  $R$  using Eq. (6).
8: Compute variance  $\sigma$  of embeddings in  $H$ .
9: Calculate adaptive radius  $R_{\text{adp}}$  according to Eq. (7).
10: for each  $\bar{E}_k$  in  $H$  do
11:   if Eq. (8) is satisfied then
12:      $S \leftarrow S \cup \{\bar{E}_k\}$ .
13:   end if
14: end for
```

expansion improve the separation between benign and poisoned embeddings. We generate 100 benign-benign and 100 benign-poisoned embedding pairs, computing their average distances as D_{Ben} and D_{Poi} , respectively. The separation ratio is defined as $\text{Separation ratio} = \frac{D_{\text{Poi}}}{D_{\text{Ben}}}$. Fig. 3 reports the separation ratios under five backdoor attacks: VILLAIN [4], Fu [17], BadVFL [32], He [19], and Adaptive attacks. “Separation ratio-E”, “-U”, and “-H” refer to calculations on embedding sets E , U , and H . Results show that while reduction yields moderate separation, expansion significantly boosts it, making poisoned embeddings more distinguishable. This highlights the effectiveness of SecureSplit in enhancing embedding discrimination, even under stealthy attacks. However, as later shown in Table 6, directly applying clustering to H remains ineffective for defense.

4.3 Adaptive Majority-based Filtering

Once H is obtained, the separation between poisoned and benign embeddings becomes more pronounced. Building on this, we propose an adaptive detection method to identify poisoned embeddings. This design is motivated by the observation that backdoor attacks in SL typically poison only a small fraction of embeddings to remain stealthy. Poisoning too many embeddings would not only increase the risk of detection but also introduce excessive manipulation costs, making the attack less feasible. Thus, most embeddings remain benign, and poisoning a majority is impractical without distorting the feature space or degrading performance. Building on this intuition, we design a majority-based filtering mechanism that retains embeddings consistent with the majority consensus. Specifically, the server first computes the coordinate-wise median [48] of all embeddings in H as:

$$\Lambda = \text{Median}(\bar{E}_1, \bar{E}_2, \dots, \bar{E}_m), \quad (5)$$

where $\text{Median}(\cdot)$ represents the coordinate-wise median function, producing $\Lambda \in \mathbb{R}^d$.

Next, we determine whether an embedding $\bar{E}_k$ is likely benign by evaluating its closeness to Λ . An embedding is considered potentially benign if it satisfies the condition $\|\bar{E}_k - \Lambda\|_2 \leq R$, where R is a radius parameter, $\|\cdot\|_2$ is the ℓ_2 norm. Conceptually, Λ serves as

the centroid of a hypersphere, and embeddings within a distance R from it are accepted as benign. To determine R , we ensure that at least half of the embeddings lie within this ball by computing:

$$R = \min \left\{ R' \mid \left| \{k \in [m] : \|\bar{E}_k - \Lambda\|_2 \leq R'\} \right| \geq \frac{m}{2} \right\}, \quad (6)$$

where $[m]$ denotes the set $\{1, 2, \dots, m\}$, and $|\cdot|$ represents the cardinality of a set. Eq. (6) ensures that R is the smallest possible value that includes at least half of the embeddings.

However, a key limitation of this filtering mechanism is its tendency to misclassify a significant number of benign embeddings as poisoned. Since the selection criterion in Eq. (6) retains only half of the embeddings, a large fraction of benign embeddings may still be excluded. To mitigate this issue, we refine our approach by slightly enlarging R to incorporate more benign embeddings while still excluding poisoned ones. Our key insight is that poisoned embeddings tend to deviate significantly from benign ones, leading to increased variance in H after an attack. Therefore, we introduce an adaptive adjustment to R based on the variance σ of embeddings in H . If σ is small, we increase R more aggressively to capture additional benign embeddings. Conversely, if σ is large, we apply a more conservative enlargement to maintain robustness against poisoning. The adaptive radius, R_{adp} , is computed as follows:

$$R_{\text{adp}} = (1 + \frac{1}{\alpha + \sigma})R, \quad (7)$$

where α is a parameter that controls the sensitivity of the adjustment. To determine whether an embedding $\bar{E}_k$ is benign, we assess its proximity to Λ . If it meets the following condition, it is classified as benign:

$$\|\bar{E}_k - \Lambda\|_2 \leq R_{\text{adp}}, \quad (8)$$

where R_{adp} is adaptively computed using Eq. (7), with R in Eq. (7) determined by Eq. (6).

By dynamically adapting the radius parameter based on the distribution of embeddings, our method effectively balances the trade-off between excluding poisoned embeddings and preserving benign ones, enhancing robustness against backdoor attacks in SL. Algorithm 1 outlines the pseudocode for our SecureSplit. In each training round, once the server receives the complete set of embeddings E from the clients, it employs a two-step approach to mitigate the effects of backdoor attacks. In the first step (Lines 3-4 of Algorithm 1), the server applies a dimensionality compression and expansion technique to improve the distinguishability of poisoned embeddings. Subsequently, the adaptive majority-based filtering mechanism is used to identify and remove potential malicious embeddings, ultimately producing the benign embedding set S for that round (Lines 6-14).

5 Experimental Evaluation

5.1 Experimental Settings

Datasets: We conducted extensive evaluations of SecureSplit across four datasets: CIFAR-10 [22], MNIST [24], CINIC-10 [9], and ImageNette [20] (Appendix B for more details).

Backdoor attacks: We examine five distinct types of backdoor poisoning attacks, comprising four previously established attacks (VILLAIN attack [4], Fu attack [17], BadVFL attack [32], and He

Table 1: Performance of defense methods is evaluated using ACC ($\uparrow$) and ASR ($\downarrow$) metrics, where higher ACC and lower ASR indicate better performance.

Attack	Defense	CIFAR-10		MNIST		CINIC-10		ImageNette	
		ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
VILLAIN attack	No defense	0.84	0.76	0.97	0.92	0.65	0.73	0.71	0.77
	TrMean	0.83	0.57	0.91	0.72	0.55	0.49	0.54	0.57
	Multi-Krum	0.77	0.48	0.87	0.59	0.49	0.45	0.47	0.49
	HDBSCAN	0.82	0.55	0.92	0.74	0.65	0.63	0.71	0.70
	DP	0.83	0.57	0.96	0.72	0.63	0.61	0.68	0.57
	MP	0.80	0.67	0.94	0.75	0.60	0.62	0.70	0.64
	ANP	0.82	0.48	0.95	0.62	0.64	0.58	0.65	0.52
	VFLIP	0.76	0.17	0.94	0.08	0.58	0.16	0.64	0.15
	SecureSplit	0.85	0.06	0.96	0.02	0.67	0.04	0.74	0.06
Fu attack	No defense	0.82	0.72	0.97	0.94	0.62	0.67	0.68	0.72
	TrMean	0.74	0.62	0.88	0.63	0.57	0.42	0.64	0.47
	Multi-Krum	0.67	0.65	0.82	0.55	0.55	0.40	0.61	0.44
	HDBSCAN	0.75	0.66	0.89	0.80	0.64	0.55	0.69	0.64
	DP	0.82	0.48	0.95	0.92	0.66	0.38	0.66	0.50
	MP	0.82	0.48	0.97	0.62	0.63	0.39	0.70	0.49
	ANP	0.80	0.33	0.94	0.48	0.62	0.31	0.67	0.39
	VFLIP	0.74	0.09	0.92	0.07	0.58	0.13	0.62	0.11
	SecureSplit	0.85	0.05	0.98	0.03	0.66	0.07	0.73	0.07
BadVFL attack	No defense	0.80	0.47	0.94	0.70	0.67	0.46	0.64	0.43
	TrMean	0.77	0.40	0.88	0.42	0.60	0.42	0.66	0.45
	Multi-Krum	0.64	0.34	0.91	0.33	0.52	0.44	0.52	0.41
	HDBSCAN	0.80	0.44	0.93	0.59	0.64	0.42	0.71	0.65
	DP	0.82	0.35	0.96	0.35	0.66	0.43	0.71	0.57
	MP	0.80	0.42	0.94	0.49	0.67	0.39	0.69	0.62
	ANP	0.78	0.30	0.94	0.17	0.64	0.37	0.65	0.48
	VFLIP	0.76	0.13	0.92	0.11	0.63	0.27	0.69	0.08
	SecureSplit	0.85	0.04	0.97	0.03	0.70	0.09	0.75	0.03
He attack	No defense	0.83	0.72	0.93	0.89	0.63	0.70	0.70	0.65
	TrMean	0.78	0.60	0.89	0.64	0.61	0.45	0.62	0.53
	Multi-Krum	0.70	0.58	0.82	0.52	0.57	0.38	0.58	0.44
	HDBSCAN	0.80	0.69	0.94	0.75	0.63	0.52	0.70	0.62
	DP	0.82	0.62	0.94	0.51	0.64	0.49	0.71	0.48
	MP	0.74	0.66	0.92	0.57	0.60	0.44	0.70	0.54
	ANP	0.77	0.57	0.87	0.39	0.62	0.42	0.64	0.43
	VFLIP	0.73	0.09	0.85	0.12	0.60	0.11	0.62	0.15
	SecureSplit	0.83	0.07	0.97	0.04	0.66	0.03	0.72	0.07
Adaptive attack	No defense	0.83	0.79	0.94	0.94	0.65	0.78	0.73	0.78
	TrMean	0.74	0.72	0.87	0.90	0.63	0.72	0.64	0.59
	Multi-Krum	0.71	0.55	0.84	0.65	0.62	0.51	0.62	0.53
	HDBSCAN	0.77	0.72	0.92	0.94	0.64	0.67	0.67	0.74
	DP	0.80	0.66	0.90	0.94	0.65	0.70	0.69	0.59
	MP	0.75	0.69	0.89	0.81	0.64	0.68	0.67	0.74
	ANP	0.79	0.58	0.92	0.68	0.65	0.66	0.67	0.59
	VFLIP	0.76	0.15	0.90	0.19	0.69	0.32	0.67	0.20
	SecureSplit	0.87	0.08	0.97	0.05	0.67	0.07	0.73	0.07

attack [19]), along with a newly designed Adaptive attack. Appendix C provides an in-depth overview of these backdoor attacks.

Compared methods: We benchmark SecureSplit against seven defenses: Trimmed-mean (TrMean) [48], Multi-Krum [5], HDBSCAN [27], differential privacy (DP) [2], model pruning (MP) [25], adversarial neuron pruning (ANP) [45], and VFLIP [8]. Appendix D contains further details.

Evaluation metrics: Two metrics are used: testing accuracy (ACC) and attack success rate (ASR). ACC represents the proportion of clean testing examples that are correctly classified, while ASR indicates the fraction of trigger-infected testing inputs predicted as the attacker’s chosen target label. A higher ACC and a lower ASR indicate better defense performance.

Non-IID setting: In SL, each client holds only a portion of the features for the entire training dataset. To simulate feature-level Non-IID heterogeneity, we allocate to each client a subset of features consisting of both shared and unique parts, regulated by an overlapping degree ρ . Adjusting ρ from 1 to 0 allows us to control

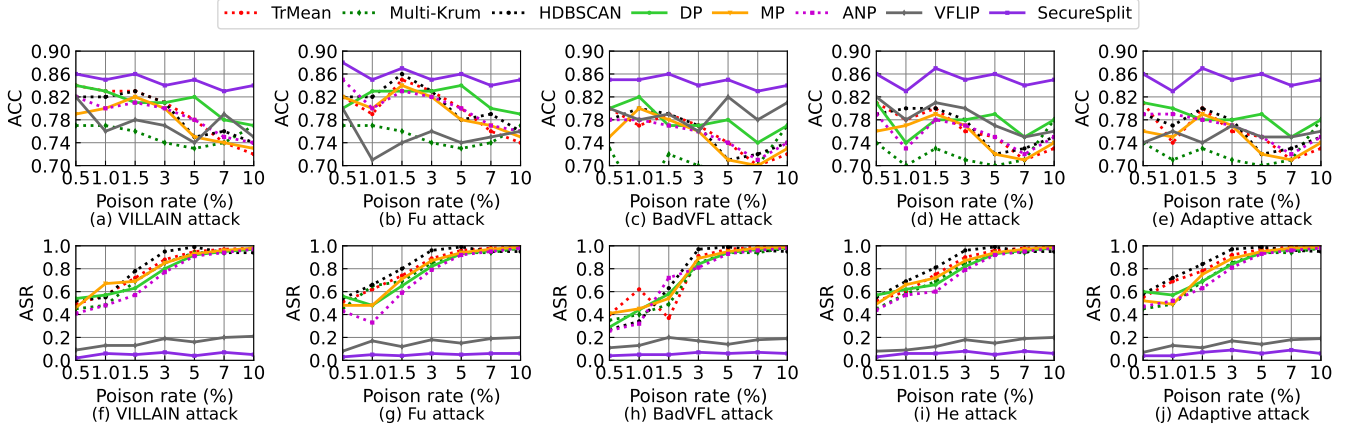


Figure 4: Impact of the poison rate, where CIFAR-10 dataset is considered.

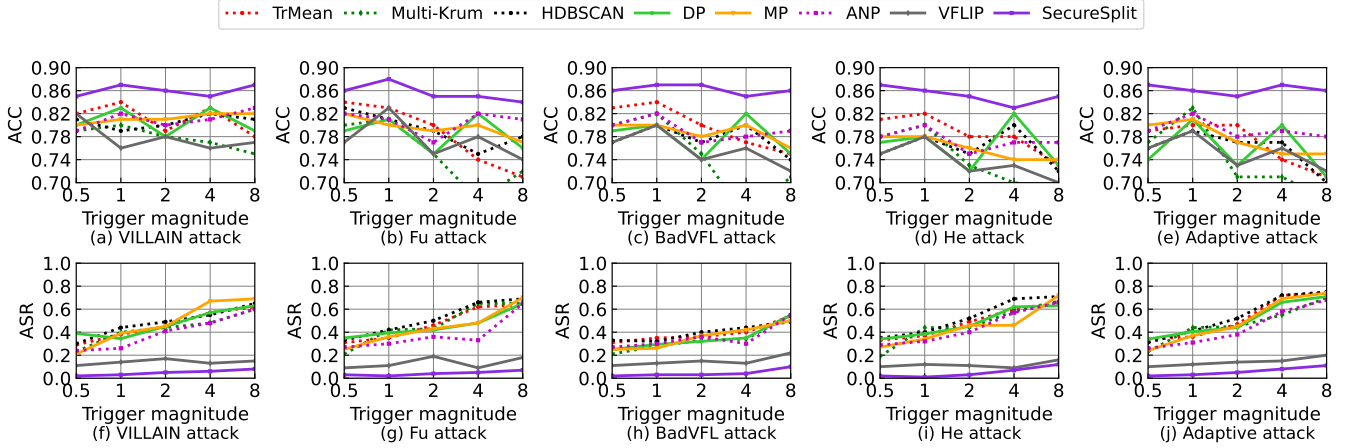


Figure 5: Impact of the trigger magnitude, where CIFAR-10 dataset is considered.

the degree of heterogeneity, ranging from fully IID ($\rho = 1$) to the most extreme Non-IID ($\rho = 0$) setting. In our experiments, we adopt the most heterogeneous case by setting $\rho = 0$, meaning clients have entirely disjoint feature subsets. See Appendix E for details.

Parameter setting: We assess SecureSplit in a setting with four participating clients, including one malicious client as in [8]. Following existing studies [4, 8, 17, 19], we randomly partition the feature space vertically across clients, ensuring no overlap among client features. By default, all clients participate in each training round. Details on the neural network architectures, learning rates, batch sizes, and total training rounds for each dataset are provided in Appendix F. For embedding aggregation, the concatenation method is used by default, and the reduced dimension d_u is set to 2. To ensure a fair comparison, the trigger size for the VILLAIN and BadVFL attacks aligns with [32], while the trigger size for the He attack follows the parameters in [19]. In the Fu attack [17], all poisoned local client embeddings are replaced with the trigger. Across all datasets, the trigger magnitude λ (the coefficient of the backdoor trigger injected into the embedding layer) is set to 4, and the poison rate (the proportion of poisoned embeddings relative to the total training embeddings) is maintained at 1% following [4].

5.2 Experimental results

SecureSplit is effective: In Table 1, we evaluate the performance of our proposed SecureSplit method alongside various defense strategies under different attack scenarios. Our results demonstrate that SecureSplit consistently outperforms other defense methods across multiple datasets and attack types, achieving the highest ACC and the lowest ASR in nearly all cases. For instance, under the VILLAIN attack on CIFAR-10, SecureSplit achieves an ACC of 0.85 and an ASR of 0.06, significantly surpassing methods like TrMean (ACC: 0.83, ASR: 0.57) and Multi-Krum (ACC: 0.77, ASR: 0.48). These results highlight the robustness of SecureSplit in mitigating adversarial attacks while preserving model accuracy, making it a superior defense mechanism for secure SL.

Impact of poison rate: Fig. 4 displays the ACC and ASR results for various defense baselines under five different attacks on the CIFAR-10 dataset, as the poisoned embedding rate increases from 0.5% to 10%. As the poison rate rises, the ASR of other defense methods steadily increases, with MP surpassing 95% ASR when the poison rate reaches 3%. In contrast, SecureSplit demonstrates exceptional robustness, keeping the ASR below 10% across all poison rates.

Impact of trigger magnitude: Fig. 5 illustrates the impact of trigger magnitude on the defense model, with values tested in the

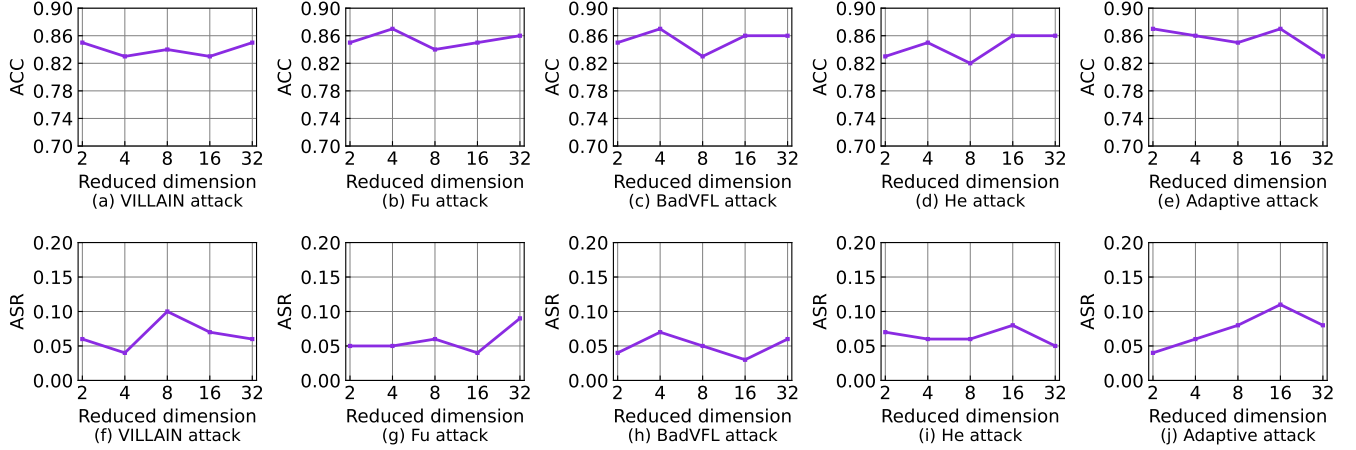


Figure 6: Impact of the reduced dimension, where the CIFAR-10 dataset is considered.

Table 2: Impact of different embedding aggregation approaches. The CIFAR-10 dataset is considered.

Aggregation	VILLAIN attack		Fu attack		BadVFL attack		He attack		Adaptive attack	
	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
Average	0.82	0.07	0.83	0.05	0.82	0.07	0.83	0.03	0.83	0.04
Max	0.87	0.04	0.82	0.04	0.83	0.08	0.84	0.06	0.86	0.06
Min	0.83	0.03	0.84	0.06	0.80	0.08	0.83	0.07	0.83	0.05
Median	0.85	0.09	0.81	0.03	0.84	0.09	0.84	0.03	0.84	0.06
Concatenate	0.85	0.06	0.85	0.05	0.85	0.04	0.83	0.07	0.87	0.08

Table 3: Performance of various defenses under multiple malicious clients. The CIFAR-10 dataset is considered.

Defense	VILLAIN attack		Fu attack		BadVFL		He attack		Adaptive attack	
	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
No defense	0.78	0.97	0.75	0.92	0.80	0.87	0.74	0.99	0.78	0.97
TrMean	0.82	0.86	0.82	0.95	0.83	0.82	0.76	0.93	0.82	0.90
Multi-Krum	0.72	0.82	0.77	0.88	0.70	0.79	0.68	0.88	0.84	0.89
HDBSCAN	0.80	0.94	0.80	0.83	0.76	0.88	0.78	0.98	0.80	0.96
DP	0.78	0.89	0.79	0.82	0.83	0.85	0.77	0.94	0.77	0.94
MP	0.79	0.85	0.82	0.84	0.79	0.78	0.80	0.94	0.81	0.96
ANP	0.72	0.88	0.78	0.87	0.81	0.73	0.78	0.92	0.82	0.92
VFLIP	0.74	0.23	0.75	0.21	0.74	0.24	0.77	0.29	0.76	0.39
SecureSplit	0.86	0.04	0.84	0.07	0.83	0.06	0.84	0.05	0.85	0.14

range of [0.5, 8]. As shown, the ASR of other methods rises with an increase in trigger magnitude. Under the He attack, when the trigger magnitude reaches 8.0, the ASR of DP approaches 85%. In contrast, our method consistently keeps the ASR below 0.10 for all tested magnitudes, highlighting its robustness.

Impact of reduced dimension d_u : Fig. 6 shows the effect of different levels of dimensionality reduction on ACC and ASR. Increasing the reduction dimension has minimal impact on the VILLAIN, Fu, and BadVFL attacks, but it negatively influences performance under He and Adaptive attacks.

Impact of different embedding aggregation approaches: Table 2 presents the performance of five embedding aggregation methods (embedding aggregation method is the $\mathcal{A}(\cdot)$ in Eq. (2)) under various attack scenarios: Average (computing the element-wise average), Max (selecting the element-wise maximum), Min (choosing the element-wise minimum), Median (picking the element-wise median), and Concatenate (combining embeddings from different clients). The experiments, conducted on the CIFAR-10 dataset, assume identical embedding dimensions across local clients, as

per [4]. Under the VILLAIN attack, the Max method yielded the highest ACC (0.87), while the Min method achieved the lowest ASR (0.03). Under the He attack, the Median method performed strongly in both ACC and ASR (0.84 and 0.03, respectively). Overall, SecureSplit offers robust defense across all aggregation methods.

Performance with multiple malicious clients: To further assess the robustness of SecureSplit, we conducted an experiment with multiple malicious clients. In this setup, eight clients participate in the training process, three of which are malicious. The experiment is carried out on CIFAR-10. This scenario, involving multiple malicious clients, presents a more challenging and aggressive setup compared to a single malicious client. Table 3 shows the performance of various defense methods in a scenario with multiple malicious clients, evaluated using ACC and ASR. SecureSplit consistently achieves the best performance, with the highest ACC and lowest ASR across all attacks.

Impact of total number of clients: Fig. 7 illustrates the performance of ACC and ASR across various attacks and different total client numbers (2, 4, 6, and 8). As the number of clients increases, the performance of all methods generally improves. In our proposed SecureSplit, the ASR metric consistently performs well, never exceeding 0.10 in any scenario.

Impact of degree of Non-IID: By default, we adopt the most extreme Non-IID scenario, where each client holds a completely disjoint set of features. Here, we analyze how varying the degree of feature overlap affects the performance of different defenses under various attacks. Results in Fig. 8 in Appendix show that our proposed SecureSplit consistently maintains strong performance across different levels of feature heterogeneity.

Impact of the number of selected clients per round: By default, we assume that all clients participate in each training round. In this section, we explore a more practical setting where only a subset of clients joins each round. For example, with four total clients, only one or two may participate in a given round. This introduces the possibility that a malicious client is selected when only one participates. As shown in Fig. 9 in Appendix, our SecureSplit remains robust under this setting.

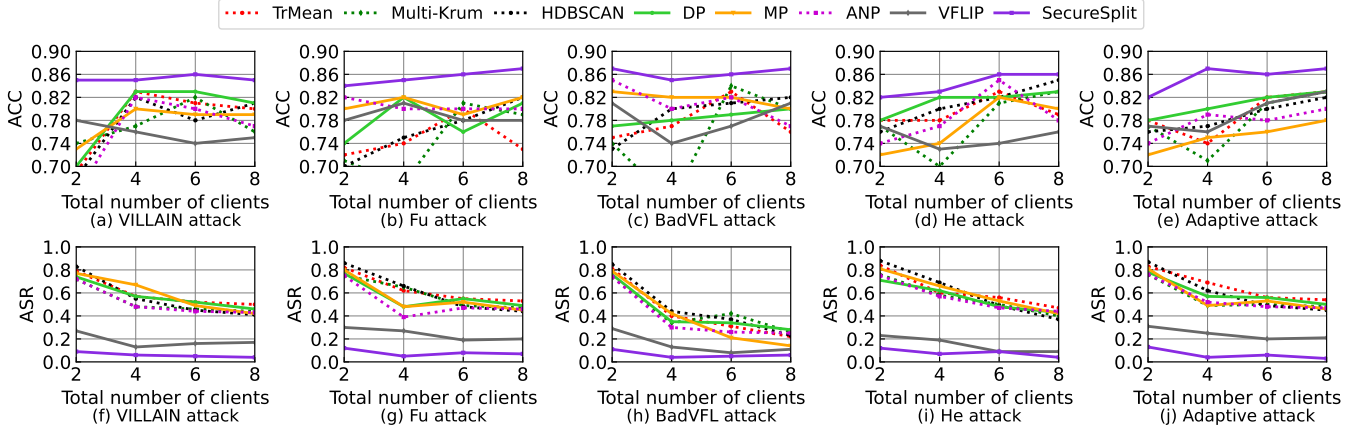


Figure 7: Impact of the total number of clients, where CIFAR-10 dataset is considered.

Table 4: Performance of various defenses under different dimensionality expansion and reduction methods. The CIFAR-10 dataset is considered.

Dimensionality transformation		VILLAIN attack		Fu attack		BadVFL attack		He attack		Adaptive attack	
		ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
Dimensionality reduction	PCA	0.82	0.25	0.84	0.07	0.82	0.03	0.83	0.22	0.83	0.09
	t-SNE	0.84	0.19	0.83	0.05	0.86	0.02	0.85	0.27	0.85	0.05
	Factor analysis	0.83	0.17	0.81	0.08	0.83	0.06	0.82	0.08	0.82	0.05
	UMAP (SecureSplit)	0.85	0.06	0.85	0.05	0.85	0.04	0.83	0.07	0.87	0.08
Dimensionality expansion	RBF	0.82	0.12	0.83	0.07	0.84	0.04	0.84	0.13	0.82	0.13
	Feature expansion	0.84	0.09	0.82	0.10	0.81	0.07	0.82	0.10	0.87	0.06
	PKT (SecureSplit)	0.85	0.06	0.85	0.05	0.85	0.04	0.83	0.07	0.87	0.08

Impact of different dimensionality expansion and reduction methods: Table 4 presents the performance of various dimensionality reduction and expansion methods in SecureSplit against five attack types on the CIFAR-10 dataset, using ACC and ASR metrics. Dimensionality reduction methods include PCA [3], t-SNE [41], and factor analysis [21], while expansion methods include RBF [37] and feature expansion [38]. The results show UMAP performs best among reduction methods, while PKT excels in expansion methods.

Impact of the poisoning start round: Table 5 in Appendix shows the effect of the starting poison round on the defense method’s performance against five different attacks. The results indicate that as the starting poison round increases, ACC values generally improve. For example, under the VILLAIN attack, ACC rises from 0.83 (at round 20) to 0.86 (at rounds 60 and 100). Overall, our method proves to be a robust defense across all considered scenarios.

Different variants of SecureSplit: To assess the effectiveness of each component, we created seven distinct variants. Specifically, Variant I uses R for filtering on embedding E with Eq. (6), selecting half of the embedding samples closest to the median point of E . Variant II uses R_{opt} for filtering on embedding E with Eq. (8), where we compute the radius R_{opt} for the embedding set E and select samples within this radius. Variant III applies filtering based on R on embedding U , while Variant IV uses R_{opt} for filtering on embedding U . Variant V filters based on R on embedding H , and Variant VI uses K-means for filtering on embedding H . These variants allow a systematic evaluation of how dimensionality reduction, expansion, and aggregation strategies affect model performance. Table 6 in Appendix clearly shows that each component is essential, and only their combination ensures robust defense.

Computational overhead of different methods: Figs. 10-13 in the Appendix present the runtime comparison of various methods across four datasets under the He attack. Computational overhead is quantified by the total execution time of each method. The results show that SecureSplit introduces only minimal additional cost relative to the baseline setting without any defense.

Compare SecureSplit with SafeSplit [35]: We do not compare SecureSplit with SafeSplit [35] in the above experiments, as SafeSplit is specifically designed for U-shaped SL with sequential client training and cannot be applied to the standard SL setting, where clients train in parallel. See Appendix G for more details on U-shaped SL. To fairly assess SecureSplit, we adapt it and other standard SL methods to the U-shaped SL setup, following the configuration in [35] with 10 clients (2 malicious) and their backdoor attack. As shown in Table 7 in Appendix, SecureSplit generalizes well to this setting and outperforms SafeSplit.

6 Conclusion

We propose SecureSplit, a new defense against backdoor attacks in SL. SecureSplit enhances detection by reshaping embeddings to highlight differences between benign and malicious inputs. It then applies a dynamic majority-based filter to identify and remove compromised embeddings. Comprehensive experiments confirm the effectiveness of SecureSplit.

Acknowledgments

We thank the anonymous reviewers for their comments.

References

- [1] [n. d.]. *Utilization of FATE in Risk Management of Credit in Small and Micro Enterprises*. <https://www.fedai.org/cases/utilization-of-fate-in-risk-management-of-credit-in-small-and-micro-enterprises/>
- [2] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep learning with differential privacy. In *CCS*.
- [3] Hervé Abdi and Lynne J Williams. 2010. Principal component analysis. *Wiley interdisciplinary reviews: computational statistics* 2, 4 (2010), 433–459.
- [4] Yijie Bai, Yanjiao Chen, Hanlei Zhang, Wenyuan Xu, Haiqin Weng, and Dou Goodman. 2023. VILLAIN: Backdoor attacks against vertical split learning. In *USENIX Security Symposium*.
- [5] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. 2017. Machine learning with adversaries: Byzantine tolerant gradient descent. In *NeurIPS*.
- [6] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. 2021. Fltrust: Byzantine-robust federated learning via trust bootstrapping. In *NDSS*.
- [7] Yiwei Chen, Kaiyu Li, Guoliang Li, and Yong Wang. 2024. Contributions Estimation in Federated Learning: A Comprehensive Experimental Evaluation. In *Vldb*.
- [8] Yungi Cho, Woorim Han, Miseon Yu, Younghun Lee, Ho Bae, and Yunheung Paek. 2024. VFLIP: A Backdoor Defense for Vertical Federated Learning via Identification and Purification. In *European Symposium on Research in Computer Security*.
- [9] Luke N Darlow, Elliot J Crowley, Antreas Antoniou, and Amos J Storkey. 2018. Cinic-10 is not imagenet or cifar-10. *arXiv preprint arXiv:1810.03505* (2018).
- [10] Zhihao Dou, Jiaqi Wang, Wei Sun, Zhuqing Liu, and Minghong Fang. 2025. Toward Malicious Clients Detection in Federated Learning. In *ASIACCS*.
- [11] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Gong. 2020. Local model poisoning attacks to Byzantine-robust federated learning. In *USENIX Security Symposium*.
- [12] Minghong Fang, Jia Liu, Neil Zhenqiang Gong, and Elizabeth S Bentley. 2022. Aflguard: Byzantine-robust asynchronous federated learning. In *ACSAC*.
- [13] Minghong Fang, Zhuqing Liu, Xuecen Zhao, and Jia Liu. 2025. Byzantine-Robust Federated Learning over Ring-All-Reduce Distributed Computing. In *The Web Conference*.
- [14] Minghong Fang, Seyedsina Nabavirazavi, Zhuqing Liu, Wei Sun, Sundararaja Sitharama Iyengar, and Haibo Yang. 2025. Do we really need to design new byzantine-robust aggregation rules?. In *NDSS*.
- [15] Minghong Fang, Xilong Wang, and Neil Zhenqiang Gong. 2025. Provably Robust Federated Reinforcement Learning. In *The Web Conference*.
- [16] Minghong Fang, Zifan Zhang, Prashant Khanduri, Jia Liu, Songtao Lu, Yuchen Liu, Neil Gong, et al. 2024. Byzantine-robust decentralized federated learning. In *CCS*.
- [17] Chong Fu, Xuhong Zhang, Shouling Ji, Jinyin Chen, Jingzheng Wu, Shanqing Guo, Jun Zhou, Alex X Liu, and Ting Wang. 2022. Label inference attacks against vertical federated learning. In *USENIX Security Symposium*.
- [18] Fangcheng Fu, Huanran Xue, Yong Cheng, Yangyu Tao, and Bin Cui. 2022. Blindfl: Vertical federated machine learning without peeking into your data. In *SIGMOD*.
- [19] Ying He, Zhili Shen, Jingyu Hua, Qixuan Dong, Jiacheng Niu, Wei Tong, Xu Huang, Chen Li, and Sheng Zhong. 2023. Backdoor attack against split neural network-based vertical federated learning. In *IEEE Transactions on Information Forensics and Security*.
- [20] Jeremy Howard and Sylvain Gugger. 2020. Fastai: a layered API for deep learning. *Information* 11, 2 (2020), 108.
- [21] Paul Kline. 2014. *An easy guide to factor analysis*. Routledge.
- [22] A. Krizhevsky and G. Hinton. 2009. Learning multiple layers of features from tiny images. *Handbook of Systemic Autoimmune Diseases* (2009).
- [23] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
- [24] Yann LeCun, Corinna Cortes, and CJ Burges. 1998. MNIST handwritten digit database. Available: <http://yann.lecun.com/exdb/mnist> (1998).
- [25] Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2018. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *RAID*.
- [26] Yang Liu, Xiong Zhang, and Libin Wang. 2020. Asymmetrical vertical federated learning. *arXiv preprint arXiv:2004.07427* (2020).
- [27] Claudia Malzer and Marcus Baum. 2020. A hybrid approach to hierarchical density-based cluster selection. In *MFI*.
- [28] Leland McInnes, John Healy, Steve Astels, et al. 2017. hdbscan: Hierarchical density based clustering. *J. Open Source Softw.* 2, 11 (2017), 205.
- [29] Leland McInnes, John Healy, and James Melville. 2018. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*.
- [30] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *AISTATS*.
- [31] Wenjin Mo, Zhiyuan Li, Minghong Fang, and Mingwei Fang. 2025. Find a Scapegoat: Poisoning Membership Inference Attack and Defense to Federated Learning. In *ICCV*.
- [32] Mohammad Naseri, Yufei Han, and Emiliano De Cristofaro. 2024. Badvfl: Backdoor attacks in vertical federated learning. In *IEEE Symposium on Security and Privacy*.
- [33] Matthias Paulik, Matt Seigel, Henry Mason, Dominic Telaar, Joris Kluivers, Rogier van Dalen, Chi Wai Lau, Luke Carlson, Filip Granqvist, Chris Vandeveld, et al. 2021. Federated evaluation and tuning for on-device personalization: System design & applications. *arXiv preprint arXiv:2102.08503* (2021).
- [34] Maarten G Poirot, Praneeth Vepakomma, Ken Chang, Jayashree Kalpathy-Cramer, Rajiv Gupta, and Ramesh Raskar. 2019. Split learning for collaborative deep learning in healthcare. *arXiv preprint arXiv:1912.12115* (2019).
- [35] Phillip Rieger, Alessandro Pegoraro, Kavita Kumari, Tigist Abera, Jonathan Knauer, and Ahmad-Reza Sadeghi. 2025. SafeSplit: A Novel Defense Against Client-Side Backdoor Attacks in Split Learning. In *NDSS*.
- [36] Daniele Romanini, Adam James Hall, Pavlos Papadopoulos, Tom Titcombe, Abbas Ismail, Tudor Cebere, Robert Sandmann, Robin Roehm, and Michael A Hoeh. 2021. Pyvertical: A vertical federated learning framework for multi-headed splitnn. *arXiv preprint arXiv:2104.00489* (2021).
- [37] Bernhard Schölkopf, Alexander Smola, and Klaus-Robert Müller. 1997. Kernel principal component analysis. In *ICANN*.
- [38] Bernhard Schölkopf and Alexander J Smola. 2002. *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press.
- [39] Abhishek Singh, Praneeth Vepakomma, Otkrist Gupta, and Ramesh Raskar. 2019. Detailed comparison of communication efficiency of split learning and federated learning. *arXiv preprint arXiv:1909.09145* (2019).
- [40] Chandra Thapa, Pathum Chamikara Mahawaga Arachchige, Seyit Camtepe, and Lichao Sun. 2022. Splitfed: When federated learning meets split learning. In *AAAI*.
- [41] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).
- [42] Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. 2018. Split learning for health: Distributed deep learning without sharing raw patient data. *arXiv preprint arXiv:1812.00564* (2018).
- [43] Wenbin Wang, Qiwen Ma, Zifan Zhang, Yuchen Liu, Zhuqing Liu, and Minghong Fang. 2025. Poisoning attacks and defenses to federated unlearning. In *The Web Conference*.
- [44] Alexander Weiße, Gerhard Wellein, Andreas Alvermann, and Holger Fehske. 2006. The kernel polynomial method. In *Reviews of modern physics*.
- [45] Dongxian Wu and Yisen Wang. 2021. Adversarial neuron pruning purifies backdoored deep models. In *NeurIPS*.
- [46] Yueqi Xie, Minghong Fang, and Neil Zhenqiang Gong. 2024. Fedredefense: Defending against model poisoning attacks for federated learning using model update reconstruction error. *ICML*.
- [47] Yuexiang Xie, Zhen Wang, Dawei Gao, Daoyuan Chen, Liuyi Yao, Weirui Kuang, Yaliang Li, Bolin Ding, and Jingren Zhou. 2022. Federatedscope: A flexible federated learning platform for heterogeneity. In *Vldb*.
- [48] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. 2018. Byzantine-robust distributed learning: Towards optimal statistical rates. In *ICML*.
- [49] Zifan Zhang, Minghong Fang, Mingzhe Chen, Gaolei Li, Xi Lin, and Yuchen Liu. 2024. Securing distributed network digital twin systems against model poisoning attacks. In *IEEE Internet of Things Journal*.

A Differences between Horizontal Federated Learning (HFL), Vertical Federated Learning (VFL), and Split Learning (SL)

a) Horizontal federated learning (HFL): In HFL, the dataset is divided by samples among various clients, where each client possesses records corresponding to different individuals but with an identical set of features. For example, in a dataset concerning customers, separate entities such as hospitals or banks might store information for different patients or clients, yet each record contains the same attributes such as age, gender, and income.

b) Vertical federated learning (VFL): VFL is applicable when data is split by features rather than by samples. In this setting, each participating client owns a distinct set of attributes for the same group of individuals. For example, consider two organizations collaborating on shared customer data: one institution may store personal details such as name, age, and gender, while another

Algorithm 2 Adaptive attack.

Input: Poisoned embedding $\bar{E}_k$, initial trigger magnitude λ_0 , median Λ of H , adaptive radius R_{adp} , and tolerance threshold ϵ_{tol} .

Output: Optimized trigger magnitude λ_{opt} .

```

1: Initialize  $\lambda_{\text{max}} \leftarrow \lambda_0, \lambda_{\text{min}} \leftarrow 0, \lambda_{\text{opt}} \leftarrow 0$ 
2: while  $\lambda_{\text{max}} - \lambda_{\text{min}} > \epsilon_{\text{tol}}$  do            $\triangleright$  Tolerance threshold for
   convergence
3:    $\lambda \leftarrow (\lambda_{\text{max}} + \lambda_{\text{min}})/2$   $\triangleright$  Binary search on trigger magnitude
4:   Compute updated poisoned embedding  $\bar{E}_k$  with  $\lambda$ 
5:   if  $\|\bar{E}_k - \Lambda\|_2 \leq R_{\text{adp}}$  then            $\triangleright$  Check defense condition
6:      $\lambda_{\text{opt}} \leftarrow \lambda$                     $\triangleright$  Update optimal trigger magnitude
7:      $\lambda_{\text{min}} \leftarrow \lambda$                     $\triangleright$  Increase lower bound
8:   else
9:      $\lambda_{\text{max}} \leftarrow \lambda$                     $\triangleright$  Decrease upper bound
10:  end if
11: end while
12: return  $\lambda_{\text{opt}}$ 

```

retains financial information like purchase records and payment history. Although the attributes differ across organizations, they correspond to the same underlying user base.

c) Split learning (SL): In SL, data is also partitioned by features rather than by samples. In this setting, each participating client holds a distinct subset of features for the same set of samples. Unlike traditional VFL, SL involves a single model that is divided between the client and the server. For example, a deep learning model may be split such that the client computes the bottom model, while the server processes the top model.

B Details of Datasets

a) CIFAR-10 [22]: It contains 60,000 color images in 10 classes, with 50,000 for training and 10,000 for testing.

b) ImageNette [20]: ImageNette is a 10-class subset of the ImageNet dataset, consisting of 9,469 training images and 3,925 testing images.

c) MNIST [24]: MNIST is a handwritten digit dataset with 10 classes, including 60,000 training and 10,000 testing images.

d) CINIC-10 [9]: CINIC-10 includes 270,000 color images in 10 classes, designed as an extension of CIFAR-10.

C Details of Backdoor Attacks

a) VILLAIN attack [4]: The VILLAIN attack is a targeted assault in which the attacker uses a label inference module to pinpoint samples with specific target labels, subsequently embedding a covert additive trigger into them.

b) He attack [19]: In this attack, the attacker leverages a small collection of labeled target-class data acquired offline and substitutes their local embeddings with a trigger vector during training, enabling control over the final model.

c) Fu attack [17]: Fu is a label inference attack aimed at deducing the labels of samples. Once the samples are identified as belonging to the target class, a conventional replacement trigger [1, -1, 1, -1, 1] is applied to create poisoned embeddings.

d) BadVFL attack [32]: BadVFL also starts with a label inference phase, where an auxiliary dataset is used to estimate training labels.

Once the source and target classes are identified, a saliency map directs the insertion of a trigger into the features of the source class, generating poisoned embeddings that are moved closer to the target class in the embedding space.

e) Adaptive attack: In the worst-case scenario, where the attacker has full knowledge of the system, specifically our SecureSplit, we devise an adaptive attack strategy targeting SecureSplit. This strategy dynamically adjusts the attack trigger magnitude, λ , and uses a binary search approach to iteratively find the maximum λ value. This ensures the attack can influence model updates effectively while evading detection. By doing so, the attacker can carry out the attack without being identified by the defense mechanism, progressively enhancing its impact. This illustrates the strategy's flexibility and adaptability in bypassing defenses. The adaptive attack for our SecureSplit is outlined in Algorithm 2 in Appendix.

D Details of Compared Methods

a) Trimmed-mean (TrMean) [48]: The server removes some largest values and smallest values from each dimension of the data, then utilizes the remaining embeddings for training.

b) Multi-Krum [5]: Upon receiving the embeddings uploaded by clients and generating the aggregated embeddings, the server selects several candidate aggregated embeddings by minimizing the total distance to their corresponding neighboring subsets.

c) HDBSCAN [27]: Once the aggregated embeddings are generated, the server constructs a density-based hierarchy and identifies stable clusters.

d) Differential privacy (DP) [2]: During the training phase, the server introduces Gaussian noise with variance σ_{noise}^2 to the aggregated embeddings. These noise-altered embeddings are subsequently used to update the top model, with σ_{noise} controlling the degree of differential privacy.

e) Model pruning (MP) [25]: The server trims the highest weights in each layer of the trained top model by setting them to zero. The pruned model is then employed for inference.

f) Adversarial neuron pruning (ANP) [45]: The server conducts adversarial pruning by perturbing the trained weights of the top model to pinpoint the most sensitive weights. The identified weights are zeroed out, and the pruned model is used for inference.

g) VFLIP [8]: VFLIP is a server-side masked auto-encoder (MAE)-based defense that cleans abnormal embeddings to mitigate backdoor attacks in SL while preserving inference accuracy.

E Details of Non-IID Simulation

To model feature-level heterogeneity across clients in SL, we incorporate a controllable parameter $\rho \in [0, 1]$ that specifies the extent to which features are shared among clients. Suppose there are n clients and a global feature space of size z . Each client C_i is assigned a subset of features denoted by $\hat{x}_k^i$, with $|\hat{x}_k^i|$ indicating the number of features allocated to that client.

The simulation begins by considering the global feature index set $\{1, 2, \dots, z\}$. Each client's local feature set is formed by dividing $|\hat{x}_k^i|$ features into two distinct parts: a shared portion common to all clients and a private portion unique to each. The shared

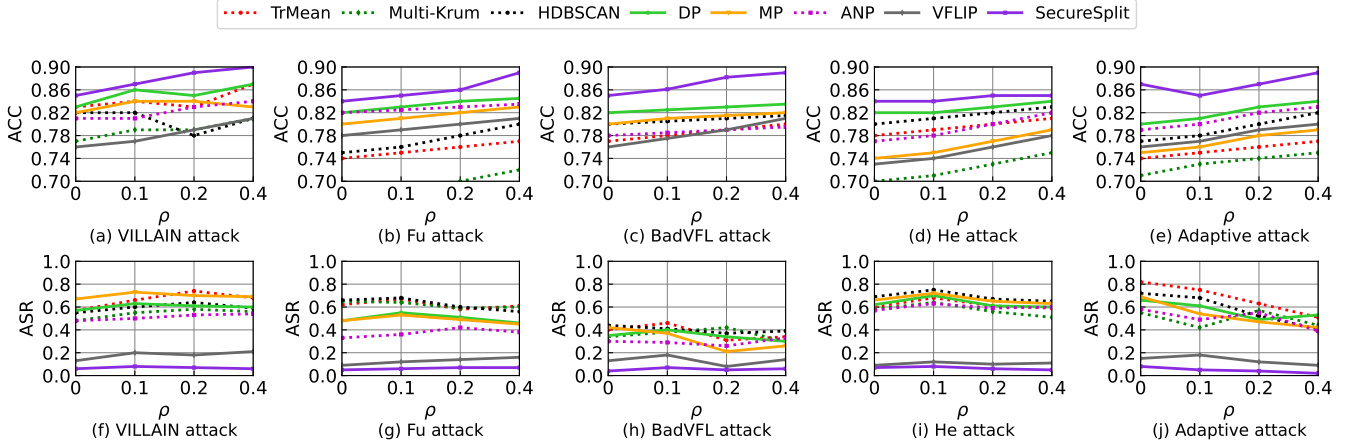


Figure 8: Impact of degree of Non-IID, where CIFAR-10 dataset is considered.

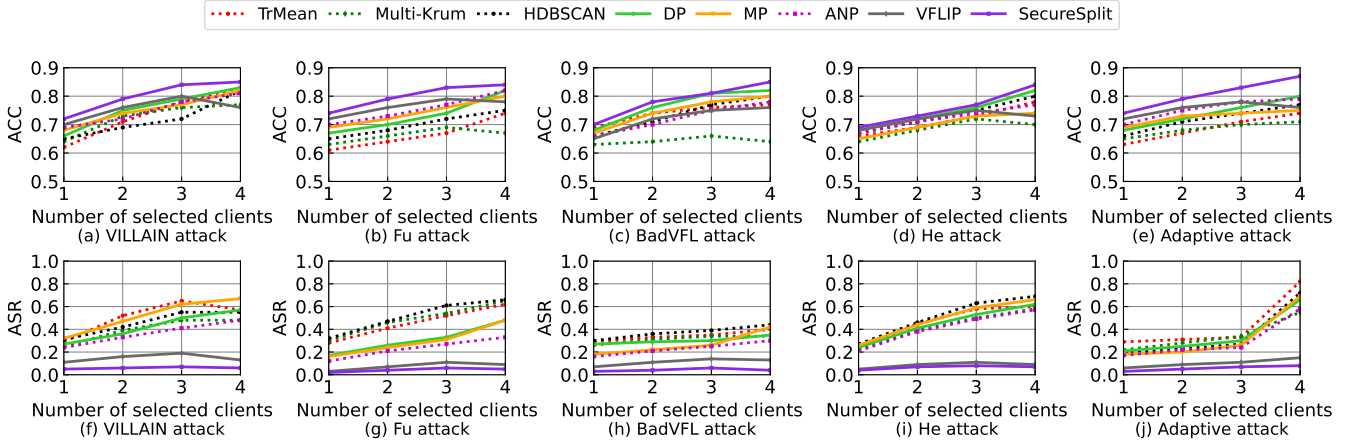


Figure 9: Impact of the number of selected clients per round, where CIFAR-10 dataset is considered.

component includes $\rho \cdot |\hat{\mathbf{x}}_k^i|$ features, randomly selected from the global feature space and assigned uniformly to every client. The remaining $(1 - \rho) \cdot |\hat{\mathbf{x}}_k^i|$ features are drawn from the unused portion of the global space and are distributed to clients in a way that prevents any overlap between them.

For each client, the final feature allocation $\hat{\mathbf{x}}_k^i$ is constructed by combining the shared features with the client-specific private features. This approach allows systematic control over how much the clients' feature sets overlap. A setting of $\rho = 0$ ensures completely disjoint feature subsets, simulating a highly Non-IID environment. In contrast, $\rho = 1$ corresponds to identical feature sets across all clients, reflecting an IID condition. Values of ρ between 0 and 1 create partially overlapping configurations, allowing fine-grained tuning of feature-level Non-IID behavior in SL experiments.

F More Details of Parameter Settings

The local client model (bottom model) is a 4-layer fully connected network (FCN) for MNIST and CIFAR-10, and a VGG-19 network for ImageNette and CINIC-10. The server-side model (top model) is a 3-layer FCN across all datasets. All models are trained for 120 rounds. Following the setup in [19], the poisoning process begins at the 80th round for all datasets. The learning rate is set to 1×10^{-3} ,

with a consistent batch size of 5000 across all datasets. In this paper, we set the hyperparameter α to 1.5 by default.

G More Details of U-shaped Split Learning

a) U-shaped split learning (U-shaped SL): The U-shaped SL paradigm presented in [35] organizes the model into three sequential components: the head, the backbone, and the tail. The head operates on the client side, where it receives input from the client's local training data and processes it into an intermediate representation. This intermediate output, often called smashed data, is then forwarded to the server.

At the server, the backbone takes over. It serves as the central computational block of the model, transforming the smashed data into a more refined feature representation. Once this processing is complete, the resulting output is passed along to the tail component.

The tail is also situated on the client side. It takes the processed features from the server, applies the final layers of the model to produce predictions, and calculates the loss if labels are available. The tail then begins the backward pass by sending gradients to the backbone on the server, enabling parameter updates throughout the model.

Table 5: Impact of the poisoning start round. The CIFAR-10 dataset is considered.

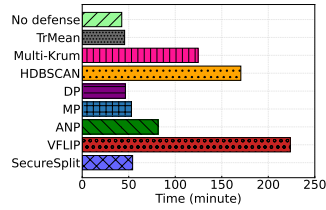
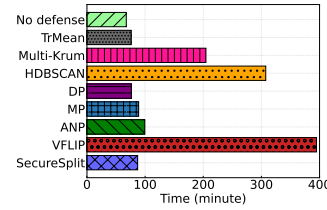
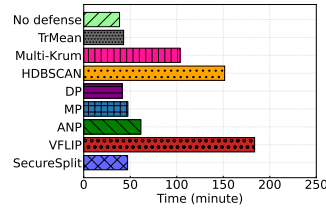
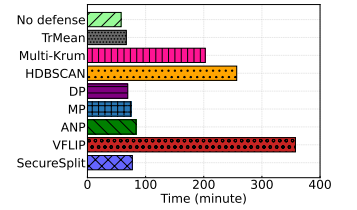
Poisoning start round	VILLAIN attack		Fu attack		BadVFL attack		He attack		Adaptive attack	
	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
20	0.83	0.07	0.82	0.07	0.84	0.06	0.82	0.05	0.82	0.07
40	0.85	0.07	0.83	0.09	0.83	0.07	0.82	0.06	0.85	0.04
60	0.86	0.06	0.83	0.07	0.84	0.06	0.84	0.07	0.84	0.03
80	0.85	0.06	0.85	0.05	0.84	0.04	0.83	0.07	0.87	0.08
100	0.86	0.03	0.86	0.03	0.86	0.03	0.85	0.02	0.86	0.07

Table 6: Different variants of SecureSplit. The CIFAR-10 dataset is considered.

Variant	VILLAIN attack		Fu attack		BadVFL attack		He attack		Adaptive attack	
	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
Variant I	0.78	0.61	0.80	0.45	0.80	0.60	0.81	0.62	0.81	0.67
Variant II	0.81	0.67	0.82	0.57	0.83	0.62	0.80	0.64	0.78	0.70
Variant III	0.80	0.49	0.76	0.42	0.80	0.42	0.78	0.51	0.80	0.44
Variant IV	0.83	0.52	0.80	0.45	0.81	0.45	0.82	0.55	0.83	0.49
Variation V	0.82	0.04	0.80	0.01	0.79	0.04	0.79	0.03	0.82	0.04
Variation VI	0.83	0.19	0.83	0.17	0.84	0.22	0.82	0.21	0.85	0.22
SecureSplit	0.85	0.06	0.85	0.05	0.85	0.04	0.83	0.07	0.87	0.08

Table 7: The performance of defense methods on various datasets with U-shape SL is evaluated using ACC ($\uparrow$) and ASR ($\downarrow$) metrics, where higher ACC and lower ASR indicate better performance.

Defense	CIFAR-10		ImageNette		MNIST		CINIC-10	
	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
No defense	0.67	0.94	0.62	0.72	0.95	0.92	0.65	0.87
TrMean	0.59	0.66	0.57	0.49	0.91	0.55	0.55	0.37
Multi-Krum	0.57	0.58	0.52	0.39	0.86	0.37	0.54	0.29
HDBSCAN	0.64	0.39	0.56	0.52	0.90	0.44	0.58	0.70
DP	0.62	0.57	0.60	0.44	0.88	0.39	0.58	0.57
MP	0.64	0.38	0.59	0.62	0.90	0.25	0.58	0.49
ANP	0.59	0.44	0.61	0.42	0.93	0.22	0.60	0.39
VFLIP	0.56	0.38	0.53	0.29	0.86	0.16	0.52	0.18
SafeSplit	0.66	0.32	0.60	0.15	0.97	0.23	0.60	0.27
SecureSplit	0.68	0.04	0.64	0.06	0.98	0.05	0.67	0.08

**Figure 10: Computation costs of different methods on CIFAR-10 dataset.****Figure 11: Computation costs of different methods on ImageNette dataset.****Figure 12: Computation costs of different methods on MNIST dataset.****Figure 13: Computation costs of different methods on CINIC-10 dataset.**

b) Adapt the SL method to U-shaped SL architectures: To reduce the impact of poisoning attacks, we incorporate defense mechanisms at the stage where the head component communicates with the backbone. This phase involves the transmission of smashed data from the client to the server, making it a critical point

for potential adversarial manipulation. To safeguard this process, we apply robust sample filtering techniques that can identify and suppress anomalous or harmful inputs before they influence the server-side computations. Examples of such techniques include TrMean, HDBSCAN, or our proposed SecureSplit.